Software Requirements Specification (SRS)

Bank Management System — Phase 1

Team E11

Meghna Sanjeev PES1UG24CS269

Mrinmayi Raman PES1UG24CS278

Advika Raj PES1UG24CS906

Manasi Vipin PES1UG24CS260

1. Introduction

1.1 Purpose

This document specifies the functional, non-functional and security requirements for a small console-based Bank Management System. It is the updated Phase-1 submission and retains the original project scope while adding the security and verification detail required by the updated deliverables.

1.2 Scope

The system supports a bank employee in creating, searching, updating and closing customer accounts, and supports a customer in depositing money, withdrawing money, checking balance and viewing a mini-statement. Account and transaction information is stored using local file handling. The project is intentionally kept simple and does not include real banking integration, online payments, networking, or an external database.

1.3 Definitions and Abbreviations

Term	Meaning
FR	Functional Requirement
NFR	Non-Functional Requirement
SEC	Security Requirement
RTM	Requirements Traceability Matrix
PIN	Personal Identification Number

2. Overall Description

2.1 Product Perspective

The product is a standalone C/C++ console application. It uses a small set of modules for the console interface, authentication, account management, transaction management and file storage. No external service is required.

2.2 Product Functions

- Employee login and administrative account management.
- Customer authentication using account number and PIN.
- Create account with customer details and initial deposit.
- Deposit and withdraw money with balance validation.
- Check balance and view recent transactions.
- Search, update and close accounts.
- Persist account and transaction information in local files.
- Display clear error messages for invalid operations.

2.3 User Classes

Actor	Responsibilities
Bank Employee	Login; create, search, update and close customer accounts.
Customer	Authenticate; deposit; withdraw; check balance; view mini-statement.

2.4 Constraints

- C/C++ console implementation.
- Local file handling instead of a database.
- No real financial transactions or external banking interfaces.
- Simple menu-driven interaction.

2.5 Assumptions

- Users provide valid basic input formats unless an error case is being tested.
- One local installation is used at a time.
- The project is an academic simulation and uses sample data only.

3. Specific Requirements

3.1 Functional Requirements

ID	Requirement
FR-1	The system shall allow an authenticated bank employee to create a customer account by entering name, address, contact number and initial deposit amount.
FR-2	The system shall generate a unique account number for each newly created account.
FR-3	The system shall allow an authenticated customer to deposit a positive amount into an existing account.
FR-4	The system shall allow an authenticated customer to withdraw a positive amount only when sufficient balance is available.
FR-5	The system shall display the current balance for an authenticated customer.
FR-6	The system shall display a mini-statement containing recent transactions for an authenticated customer.
FR-7	The system shall allow an authenticated employee to

	search for an account using its account number.
FR-8	The system shall allow an authenticated employee to update customer account details.
FR-9	The system shall allow an authenticated employee to close an existing account.
FR-10	The system shall authenticate an employee using a username and password before administrative operations.
FR-11	The system shall authenticate a customer using an account number and PIN before self-service transactions.
FR-12	The system shall display an appropriate error message when an operation fails because of invalid input, invalid credentials, insufficient balance, duplicate account data, or a non-existent account.
FR-13	The system shall store account and transaction data in local files so that data remains available after the program is restarted.

3.2 Non-Functional Requirements

ID	Category	Measurable Requirement
NFR-1	Performance	For normal local use, deposit, withdrawal and balance enquiry shall complete within 2 seconds.
NFR-2	Usability	The system shall use a numbered, menu-driven console interface with clear prompts and messages.
NFR-3	Reliability	Invalid input shall be rejected without changing the affected account balance or stored account record.
NFR-4	Maintainability	Each major feature shall be implemented in a separate function/module so that individual features can be modified independently.
NFR-5	Portability	The program shall compile using a standard C/C++ compiler on the target development OS without requiring an external service.
NFR-6	Data persistence	A successful account or transaction update shall remain available after closing and restarting the program.

3.3 Security Objectives

- SO-1: Ensure that only authenticated employees can perform administrative account operations.
- SO-2: Ensure that only an authenticated customer can access that customer's self-service banking operations.

3.4 Security Requirements

ID	Security Requirement
SEC-1	Employee authentication shall be required before create, search, update or close account operations.
SEC-2	Customer authentication shall require a valid

	account number and PIN before deposit, withdrawal, balance or mini-statement operations.
SEC-3	Failed authentication shall deny access and display a clear error message without performing the requested operation.
SEC-4	The system shall validate transaction amounts so that zero or negative deposits/withdrawals are not accepted.

4. External Interface Requirements

4.1 User Interface

The interface is a console menu. The program presents numbered choices, accepts keyboard input and displays success or error messages. No graphical UI is required.

4.2 Data Interface

Persistent data is accessed through local files. The exact file format may be chosen during implementation; it only needs to support reliable reading and writing of account and transaction records.

4.3 Software Interfaces

No external API, REST service, cloud service or database is required. The architecture uses internal C/C++ module interfaces only.

5. Use Case Model

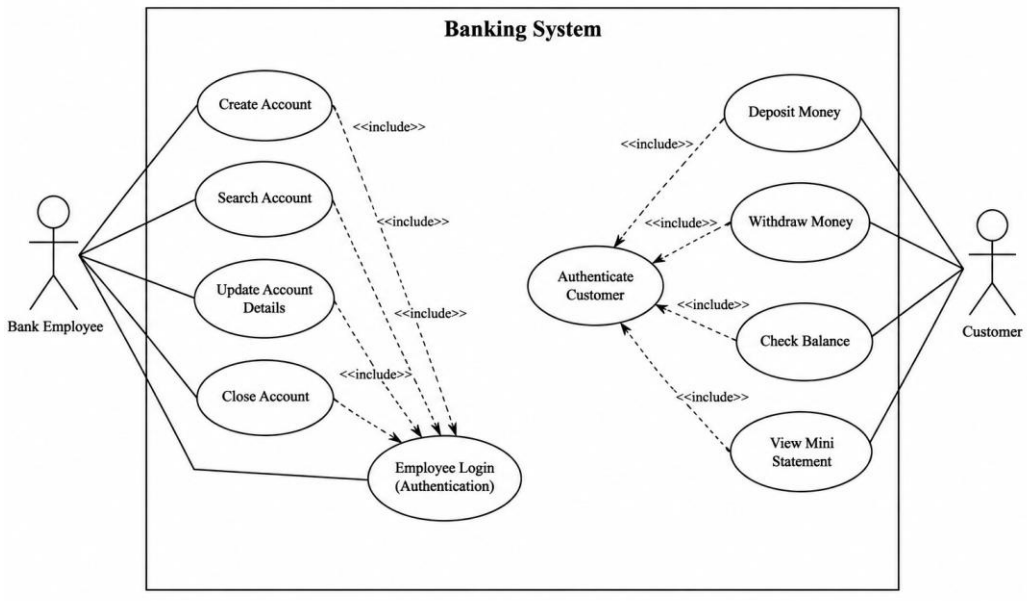


Figure 1. Use case diagram

5.1 Use Case Summary

Actor	Use Cases
Bank Employee	Employee Login; Create Account; Search Account; Update Account Details; Close Account
Customer	Authenticate Customer; Deposit Money; Withdraw Money; Check Balance; View Mini Statement

6. Requirements Traceability Matrix

Req ID	Requirement Summary	Use Case	Verification	Priority
FR-1	Create account	Create Account	TC-01, TC-02	High
FR-2	Unique account number	Create Account	TC-01	High
FR-3	Deposit	Deposit Money	TC-03, TC-04	High
FR-4	Withdrawal with balance check	Withdraw Money	TC-05, TC-06	High
FR-5	Check balance	Check Balance	TC-07	High
FR-6	Mini-statement	View Mini Statement	TC-08	Medium
FR-7	Search account	Search Account	TC-09	Medium
FR-8	Update account	Update Account Details	TC-10	Medium
FR-9	Close account	Close Account	TC-11	Medium
FR-10	Employee authentication	Employee Login	TC-12	High
FR-11	Customer authentication	Authenticate Customer	TC-13	High
FR-12	Error handling	All use cases	TC-04, TC-06, TC-14	Medium
FR-13	Persistent storage	All data operations	TC-15	High
NFR-1	≤2 s local transaction response	All transactions	TC-16	Medium
NFR-2	Simple console UI	All use cases	TC-17	Medium
NFR-3	Invalid input does not alter records	All input validation	TC-14	High
NFR-4	Modular implementation	Code structure	Code review	Low
NFR-5	Compile on target OS	Build	Build check	Low
NFR-6	Persistence across restart	All data operations	TC-15	High
SEC-1	Employee access control	Employee Login	TC-12	High
SEC-2	Customer access control	Authenticate Customer	TC-13	High
SEC-3	Failed login denies operation	Authentication	TC-12, TC-13	High
SEC-4	Positive transaction amounts	Deposit/Withdraw	TC-04, TC-06	High